

Panduan Pertanyaan Sidang Skripsi SIPEKA

Latihan jawaban tentang penelitian dan alur sistem

Panduan ini membantu Anda menjelaskan alasan penelitian, arsitektur SIPEKA, perjalanan request, pengujian API, dan keterbatasan implementasi. Gunakan contoh jawaban untuk latihan berbicara dengan bahasa sendiri. Pertanyaan berikut merupakan bahan latihan, bukan kepastian pertanyaan yang akan diajukan penguji.

Cara belajar. Baca satu pertanyaan, tutup jawabannya, lalu jelaskan selama 30 sampai 60 detik. Buka kode atau hasil uji untuk mendukung jawaban. Tandai pertanyaan yang belum bisa dijelaskan tanpa membaca.

Data penelitian. Isi lembar persiapan pada bagian akhir dengan angka dan prosedur yang benar-benar digunakan. Bedakan kemampuan arsitektur, fitur yang sudah diimplementasikan, dan hasil yang sudah diuji.

Kedudukan penelitian dan pengujian

Gunakan kerangka penerapan microservices sebagai fokus dan monolith sebagai baseline hanya jika sesuai dengan judul, rumusan masalah, tujuan, dan metodologi skripsi. Pengujian tetap memiliki unsur komparatif. Istilah baseline menjelaskan kedudukannya sebagai acuan, bukan menghilangkan fakta bahwa dua implementasi dibandingkan.

Jika ditanya mengapa judulnya bukan perbandingan arsitektur

“Objek utama penelitian saya adalah penerapan microservices pada SIPEKA. Implementasi monolith digunakan untuk memberi acuan dalam menilai hasil pengujian. Karena itu judul menekankan penerapan, sedangkan pengujian komparatif ditempatkan sebagai bagian evaluasinya.”

Bukti yang perlu ditunjukkan: keterkaitan judul dengan rumusan masalah, rancangan pembagian service, implementasi komunikasi antarservice, serta hasil pengujian. Jika seluruh rumusan masalah dan kesimpulan justru menilai arsitektur mana yang unggul, kesesuaian fokus penelitian perlu dibahas bersama pembimbing.

1 Pertanyaan pembuka penelitian

Apa masalah utama yang ingin Anda selesaikan?

Jawaban harus memuat:

- Masalah pada proses pelaporan/penanganan kasus kekerasan.
- Kebutuhan integrasi laporan masyarakat dengan proses penanganan petugas.
- Alasan penerapan microservices pada SIPEKA dan kedudukan monolith sebagai baseline evaluasi.
- Indikator evaluasi yang sesuai dengan tujuan dan rumusan masalah penelitian.

Jangan hanya menjawab:

“Saya membuat website pelaporan kekerasan.”

Sampaikan sebagai penelitian:

“Fokus penelitian saya adalah penerapan arsitektur microservices pada SIPEKA untuk mengintegrasikan pelaporan masyarakat dan penanganan kasus. Monolith digunakan sebagai baseline, yaitu acuan untuk mengevaluasi performa implementasi pada skenario pengujian yang ditentukan.”

Apa kontribusi penelitian Anda?

Pisahkan dua kontribusi:

- Kontribusi praktis: sistem pelaporan dan penanganan kasus.
- Kontribusi penelitian: rancangan pembagian layanan dan integrasi data pada SIPEKA, disertai evaluasi implementasi dengan monolith sebagai baseline. Klaim kebaruan harus dijelaskan terhadap penelitian terdahulu.

Mengapa topik ini penting?

Hubungkan dengan:

- Kecepatan laporan diterima.
- Ketertelusuran status.
- Pemisahan tugas antarlayanan.
- Kinerja implementasi pada beban yang diuji. Jangan menyimpulkan kemampuan scaling jika eksperimen scaling belum dilakukan.

2 Metodologi evaluasi implementasi

Bagaimana memastikan perbandingan microservices dan monolith adil?

Siapkan bukti konfigurasi dan catatan pelaksanaan untuk menjawab pertanyaan ini.

Kerangka jawaban yang perlu disesuaikan dengan pelaksanaan pengujian:

“Saya membandingkan fungsi dan beban yang setara, lalu menjelaskan konfigurasi perangkat keras, database, timeout, serta resource setiap arsitektur. Saya melaporkan perbedaan konfigurasi yang masih dapat memengaruhi hasil.”

Catatan: batas per container tidak sama dengan batas total arsitektur. Pada compose dasar, dua container backend microservices masing-masing dibatasi 1 CPU dan 512 MB, sedangkan monolith satu container dengan batas yang sama. Ini adalah batas maksimum, bukan jaminan pemakaian aktual. Jelaskan konfigurasi yang benar-benar digunakan saat eksperimen; jangan mengklaim resource total sama tanpa bukti.

Anda harus siap menjelaskan:

- Jumlah virtual user/thread.
- Ramp-up period.
- Durasi atau loop pengujian.
- Berapa kali pengujian diulang.
- Endpoint apa yang diuji.
- Spesifikasi komputer/server.
- Apakah database dibersihkan sebelum pengujian.
- Apakah ada warm-up.
- Apakah urutan pengujian diacak atau selalu sama.
- Metrik apa yang dikumpulkan.

Kenapa menggunakan metrik tersebut?

Biasanya:

- Response time: waktu dari request dikirim sampai respons diterima.
- Throughput: jumlah request yang diproses per satuan waktu menurut definisi alat ukur. Pada laporan JMeter, throughput dapat mencakup sampel gagal; baca bersama error rate dan jangan otomatis menyebutnya throughput sukses.
- Error rate: persentase request gagal.
- CPU dan memory: penggunaan sumber daya.
- Persentil seperti P90/P95: memperlihatkan pengalaman request lambat, bukan hanya rata-rata.

Jangan hanya memahami rumus. Anda harus bisa menjelaskan maknanya.

Contoh:

“Rata-rata response time bisa tertutupi oleh sebagian kecil request yang sangat lambat. Karena itu persentil, misalnya P95, diperlukan untuk melihat batas waktu respons yang dialami 95% request.”

Mengapa hasil microservices bisa lebih lambat atau hanya berbeda sedikit?

Jawaban:

- Ada biaya komunikasi jaringan antarservice pada operasi yang memang melibatkan lebih dari satu service.
- Ada serialisasi dan deserialisasi data.
- Periksa jalur request yang benar-benar diuji. Pada konfigurasi ini, microservices dan monolith sama-sama melewati gateway; keberadaan gateway saja tidak menjelaskan selisih performa.
- Sinkronisasi tertentu melewati message broker.
- Biaya koneksi dan pembagian resource dapat berpengaruh. Penyebab hasil harus dibuktikan dengan pengukuran, bukan hanya diduga dari nama arsitektur.

Kalau microservices lebih lambat, mengapa tetap digunakan?

“Microservices tidak selalu dipilih untuk mendapatkan response time paling kecil. Keunggulannya berada pada pemisahan tanggung jawab, isolasi kegagalan, kemudahan deployment per service, serta kemampuan scaling service tertentu secara independen. Karena itu hasil performa harus dibaca bersama trade-off arsitekturnya.”

Apakah hasil penelitian dapat digeneralisasi?

Jawaban yang aman:

“Generalisasinya terbatas pada karakteristik aplikasi, skenario beban, konfigurasi, dan lingkungan pengujian yang digunakan. Hasilnya tidak berarti microservices selalu lebih cepat atau monolith selalu lebih baik untuk semua sistem.”

Jangan mengatakan hasil Anda berlaku mutlak untuk semua aplikasi.

3 Pertanyaan tentang pemilihan arsitektur

Mengapa menggunakan microservices?

Jawaban berdasarkan proyek:

- Reporting Service mengelola laporan masyarakat.
- Case Service mengelola autentikasi dan penanganan kasus.
- Masing-masing mempunyai tanggung jawab dan data privat.
- Service dapat dikembangkan atau di-scale secara terpisah.

Mengapa hanya dua microservice?

“Pemisahan dilakukan berdasarkan domain bisnis yang jelas, bukan sekadar memperbanyak service. Pelaporan dan penanganan memiliki data serta siklus proses berbeda. Jika dipecah terlalu kecil, kompleksitas jaringan dan operasional justru bertambah tanpa manfaat yang sebanding.”

Bukankah autentikasi seharusnya service tersendiri?

Saat ini autentikasi berada dalam Case Service. Jawaban jujurnya:

“Dalam implementasi penelitian ini, autentikasi digabung ke Case Service untuk membatasi kompleksitas dan jumlah service. Pada pengembangan skala lebih besar, autentikasi dapat dipisahkan menjadi Identity Service, terutama jika dipakai oleh banyak sistem.”

Mengapa hanya satu container MongoDB?

“Database-per-service berarti setiap service memiliki kepemilikan data yang terpisah, bukan harus selalu menggunakan server fisik berbeda. Dalam proyek ini Reporting Service menggunakan report_db, sedangkan Case Service menggunakan case_db, walaupun keduanya berada dalam satu instance MongoDB.”

Kenapa tidak melakukan join antardatabase?

“Karena masing-masing service memiliki kepemilikan datanya sendiri. Case Service hanya menyimpan referensi laporan_id dan kode_laporan. Jika service lain mengakses database secara langsung, coupling antarlayanan menjadi tinggi.”

4 Pertanyaan tentang API Gateway

Siapkan jawaban untuk:

- Apa fungsi API Gateway?
- Mengapa frontend tidak langsung mengakses service?
- Bagaimana gateway menentukan service tujuan?
- Apakah gateway menjalankan logika bisnis?
- Apa yang terjadi jika gateway mati?

Jawaban inti:

“API Gateway adalah single entry point. Nginx meneruskan /api/laporan ke Reporting Service, sedangkan /api/auth dan /api/penanganan ke Case Service. Gateway tidak menyimpan data atau menjalankan logika bisnis.”

Jika gateway mati:

“Frontend tidak dapat menjangkau backend melalui alamat publik, walaupun service internal mungkin masih hidup. Karena konfigurasi saat ini hanya memiliki satu instance gateway, gateway masih menjadi single point of failure pada jalur akses melalui gateway.”

Jangan mengklaim sistem sudah high availability karena belum ada replika atau load balancer berlapis.

5 Pertanyaan alur pengiriman laporan

Anda harus dapat menjelaskan tanpa melihat kode:

Pengguna isi form → React membuat FormData → laporanService → Axios/apiClient → Nginx API Gateway → route Reporting Service → Multer menyimpan file → laporanController.store() → Laporan.create() → MongoDB report_db → kode laporan dikembalikan

Pertanyaan turunannya:

Mengapa menggunakan FormData?

Karena request dapat berisi teks dan file bukti.

Di mana file disimpan?

backend/reporting-service/storage/bukti/

Berapa ukuran maksimal file?

5 MB berdasarkan konfigurasi Multer.

Siapa yang membuat kode laporan?

Middleware pre('save') pada model Laporan.

Apa status awal laporan?

menunggu_registrasi

Apa beda kode LA dan LP?

- LA: korban anak, usia di bawah 18 tahun.
- LP: tipe laporan perempuan. Frontend memilih tipe berdasarkan usia; model backend membuat awalan kode berdasarkan tipe_laporan yang diterima.

6 Pertanyaan RabbitMQ

Untuk apa RabbitMQ?

“RabbitMQ digunakan untuk mengirim event perubahan status dari Case Service ke Reporting Service secara asynchronous. Case Service tidak perlu mengubah database milik Reporting Service secara langsung.”

Apakah pengiriman laporan memakai RabbitMQ?

Jawaban: tidak.

Pengiriman laporan menggunakan HTTP sinkron dan langsung disimpan ke report_db. RabbitMQ baru digunakan saat status kasus berubah:

- Registrasi → proses_assessment.
- Intervensi → dalam_penanganan.
- Terminasi → selesai.

Apa nama queue-nya?

kasus_status_updates

Apa yang terjadi jika Reporting Service mati?

Jika pesan sudah diterima dan tersimpan oleh broker yang tetap tersedia, pesan dapat menunggu sampai consumer Reporting Service aktif kembali. Queue menggunakan durable dan pesan menggunakan persistent.

Tetapi jawaban Anda harus hati-hati:

“Durable dan persistent membantu ketahanan pesan. Namun jika RabbitMQ tidak dapat dihubungi saat publish, pesan belum tentu masuk ke antrian. Pada kode saat ini, kegagalan publish dicatat sementara perubahan kasus tetap dapat tersimpan. Belum ada transactional outbox dan publisher confirms untuk menutup celah tersebut.”

Mengapa status belum langsung berubah?

Karena komunikasi RabbitMQ bersifat asynchronous dan menerapkan eventual consistency.

Apa itu eventual consistency?

“Data di kedua service dapat berbeda sementara. Setelah event berhasil dikirim dan diproses, status di Reporting Service menyusul status di Case Service. Jika event gagal dikirim, diperlukan pemulihan atau pengiriman ulang agar konsistensi tercapai.”

7 Pertanyaan database dan data

Mengapa menggunakan MongoDB?

Siapkan alasan yang benar-benar sesuai penelitian, misalnya:

- Struktur dokumen cocok dengan laporan dan activity_log.
- Integrasi langsung dengan Node.js dan Mongoose.
- Skema dapat berkembang.
- Tidak memerlukan relasi kompleks seperti pada sistem transaksi finansial.

Jangan mengatakan MongoDB “selalu lebih cepat” tanpa bukti pengujian.

Apa hubungan Laporan dengan Kasus?

Tidak menggunakan foreign key database. Case Service menyimpan:

laporan_id

kode_laporan

laporan_id menunjuk identitas laporan pada Reporting Service.

Bagaimana mencegah satu laporan diregistrasikan dua kali?

Controller memeriksa:

```
Kasus.findOne({ laporan_id })
```

Namun bersiap jika penguji lebih teknis:

Pemeriksaan aplikasi masih berpotensi mengalami race condition jika dua request identik masuk bersamaan. Indeks laporan_id saat ini belum dinyatakan unique. Perbaikan yang lebih kuat adalah unique index dan penanganan duplicate-key error.

Jelaskan pemeriksaan duplikat yang sudah ada serta keterbatasannya pada request bersamaan.

8 Pertanyaan keamanan

Kemungkinan pertanyaan:

- Bagaimana autentikasi bekerja?
- Apa fungsi JWT?
- Apa beda 401 dan 403?
- Bagaimana data korban dilindungi?
- Apakah password disimpan langsung?
- Apakah endpoint publik membocorkan identitas?
- Bagaimana upload file diamankan?

Jawaban dasar:

- Password di-hash menggunakan bcrypt.
- Login menghasilkan JWT.
- JWT dikirim melalui header Authorization: Bearer.
- 401 berarti belum/tidak berhasil diautentikasi.
- 403 berarti sudah dikenali tetapi tidak mempunyai role yang sesuai.
- Public GIS mengecualikan nama, NIK, dan telepon, tetapi data lokasi serta waktu kejadian tetap perlu dievaluasi karena dapat membantu identifikasi tidak langsung.

Tetapi Anda perlu mengakui keterbatasan:

- Validasi upload saat ini terutama membatasi ukuran; belum terlihat validasi ketat MIME type.
- File disimpan pada local storage container/host, belum object storage.
- Secret JWT default masih tersedia dalam konfigurasi.
- Endpoint cek status menampilkan nama_korban; ini perlu dipertimbangkan dari sisi privasi karena hanya bergantung pada kode laporan.
- Status laporan di model belum menggunakan enum.
- Belum terlihat rate limiting pada login dan endpoint publik.

Jelaskan kontrol yang sudah ada beserta keterbatasannya; hindari klaim bahwa sistem sepenuhnya aman.

9 Pertanyaan tentang validasi

Validasi dilakukan di frontend atau backend?

Keduanya, tetapi belum seluruhnya sama.

- Frontend memeriksa kronologi minimal 50 karakter dan aturan pelapor korban anak.
- Backend memvalidasi field wajib melalui Mongoose.
- Backend mengubah string Boolean dari FormData.
- Enum digunakan pada beberapa field.

Catatan penting:

Validasi frontend hanya untuk pengalaman pengguna dan dapat dilewati menggunakan Postman. Karena itu aturan bisnis penting seharusnya tetap divalidasi di backend.

Jika penguji mencoba kronologi pendek melalui Postman, backend saat ini kemungkinan tetap menerimanya selama field tersebut tidak kosong. Ini adalah potensi temuan.

10 Pertanyaan tentang alur penanganan

Hafalkan urutan ini:

Pengaduan → Registrasi → Assessment → Rencana intervensi → Monitoring → Terminasi

Endpoint-nya:

POST /api/laporan

POST /api/penanganan/registrasi

PUT /api/penanganan/:id/assessment

PUT /api/penanganan/:id/intervensi

POST /api/penanganan/:id/log

PUT /api/penanganan/:id/selesai

Pertanyaan yang mungkin muncul:

- Apakah tahap dapat dilompati?
- Apakah kasus selesai dapat diubah kembali?
- Siapa yang boleh melakukan terminasi?
- Bagaimana audit trail disimpan?
- Bagaimana jika petugas salah memasukkan assessment?

Kelemahan yang perlu diketahui: controller saat ini belum terlihat memvalidasi transisi status secara ketat. Secara API, pengguna terautentikasi kemungkinan dapat memanggil endpoint tahap tertentu secara langsung, walaupun UI mengarahkan urutan normal.

11 Pertanyaan pengujian Postman

Penguji bisa meminta demonstrasi:

1. Kirim laporan menggunakan form-data.
2. Tunjukkan respons 201.
3. Simpan kode laporan.
4. Cek status laporan tanpa login.
5. Login sebagai petugas.
6. Ambil JWT.
7. Ambil _id laporan.
8. Registrasikan laporan.
9. Ambil _id kasus.
10. Jalankan assessment, intervensi, log, dan terminasi.
11. Cek perubahan status.
12. Coba request tanpa token untuk memperlihatkan respons 401.

Siapkan juga pengujian negatif:

- Field wajib kosong → 422.
- Token tidak ada → 401.
- Token tidak valid → 401.
- Role salah → 403.
- ID berformat valid tetapi dokumennya tidak ditemukan → 404. ID dengan format tidak valid dapat menghasilkan 500 pada beberapa controller saat ini.
- Registrasi laporan yang sama dua kali → 422.
- File lebih dari 5 MB → upload gagal.

12 Jawaban sepuluh pertanyaan prioritas

Gunakan contoh jawaban berikut dengan bahasa sendiri. Sesuaikan pernyataan pelaksanaan dan hasil dengan bukti dalam skripsi; jangan membaca rencana pengujian seolah-olah sudah dilakukan.

1. Apa novelty atau kontribusi penelitian Anda?

“Kontribusi penelitian saya adalah rancangan dan penerapan microservices pada SIPEKA, dengan pemisahan layanan pelaporan dan penanganan kasus, kepemilikan data per layanan, serta sinkronisasi status melalui RabbitMQ. Saya juga mengevaluasi performa implementasinya menggunakan monolith sebagai baseline. Kontribusi praktisnya adalah integrasi alur pelaporan masyarakat sampai terminasi kasus.”

Catatan pendalaman. Kontribusi belum otomatis berarti kebaruan ilmiah. Untuk menjawab novelty, tunjukkan perbedaan spesifik dengan penelitian terdahulu yang benar-benar ditelaah. Jangan mengklaim pertama, algoritma baru, atau performa lebih unggul tanpa bukti.

2. Mengapa membandingkan microservices dengan monolith?

“Monolith saya gunakan sebagai baseline agar pengukuran performa microservices memiliki acuan pada fungsi aplikasi yang setara. Tujuan utamanya tetap mengevaluasi penerapan microservices di SIPEKA. Perbandingan ini membantu melihat konsekuensi implementasi, termasuk jika hasilnya hanya berbeda sedikit atau microservices lebih lambat.”

Catatan pendalaman. Jika penguji mempertanyakan judul, jelaskan hubungan tujuan penerapan dengan evaluasinya. Jangan menyangkal adanya perbandingan dan jangan mengubah tujuan penelitian hanya pada jawaban lisan.

3. Bagaimana menjamin perbandingannya adil?

“Saya harus menjelaskan kesetaraan fungsi endpoint, payload, kondisi awal data, beban, durasi, timeout, dan lingkungan pengujian. Saya juga melaporkan batas resource per container dan total setiap arsitektur. Keadilan pengujian dinilai dari bukti konfigurasi dan pelaksanaannya, sehingga saya tidak menyatakan semuanya setara jika masih ada perbedaan.”

Catatan pendalaman. Pada compose dasar, dua backend microservices masing-masing dibatasi 1 CPU dan 512 MB, sedangkan monolith satu backend dengan batas yang sama. Total batas backend berbeda; database, broker, dan gateway juga perlu diperhitungkan sesuai cakupan evaluasi. File pengujian menunjuk salinan database, tetapi tidak meresetnya otomatis setelah setiap run. Gunakan konfigurasi yang benar-benar dipakai dan akui potensi faktor perancu.

4. Apa kesimpulan yang benar-benar didukung oleh hasil pengujian?

“Jika tabel hasil menunjukkan selisih relatif kecil, kesimpulan saya adalah bahwa pada beban dan konfigurasi yang diuji, perbedaan performa yang teramati juga relatif kecil. Saya tidak menyimpulkan microservices selalu lebih cepat. Keberhasilan fungsi sistem hanya saya nyatakan untuk skenario fungsional yang telah diuji dan lulus.”

Catatan pendalaman. Isi dengan nilai aktual response time, throughput, error rate, dan variasi antarulang. Tanpa analisis yang memadai, jangan menyebut kedua arsitektur setara secara statistik atau tidak berbeda signifikan. Selisih kecil tetap sah; angka, grafik, dan log harus tetap sesuai hasil pengukuran.

5. Mengapa memakai microservices jika performanya belum tentu lebih baik?

“Pertimbangan saya mencakup pemisahan tanggung jawab antara pelaporan dan penanganan kasus serta pengelolaan layanan secara terpisah. Keputusan arsitektur mempunyai konsekuensi biaya komunikasi dan operasional. Jika selisih performanya kecil, saya tetap melaporkan hasil tersebut dan menilai pilihan arsitektur berdasarkan kebutuhan sistem, bukan hanya kecepatan respons.”

Catatan pendalaman. Scaling independen dan isolasi kegagalan adalah potensi desain yang perlu dibuktikan melalui eksperimen tersendiri. Monolith juga dapat menjadi pilihan wajar untuk sistem kecil; jangan mengklaim microservices selalu lebih tepat.

6. Mengapa hanya dua service?

“Saya membagi layanan berdasarkan tanggung jawab bisnis. Reporting Service menerima dan menyimpan laporan masyarakat. Case Service menangani registrasi, assessment, intervensi, monitoring, dan terminasi. Autentikasi masih digabung di Case Service untuk membatasi kompleksitas implementasi. Jumlah service mengikuti kebutuhan domain, bukan semakin banyak semakin baik.”

Catatan pendalaman. Tunjukkan route serta database masing-masing. Jika kebutuhan berkembang, pemisahan autentikasi dapat dipertimbangkan dengan memperhatikan manfaat dan tambahan biaya operasionalnya.

7. Mengapa RabbitMQ diperlukan?

“RabbitMQ dipilih untuk menyampaikan event perubahan status dari Case Service ke Reporting Service secara asynchronous. Case Service menyimpan penanganan pada databasenya, lalu Reporting Service memperbarui status laporan setelah menerima event. Dengan begitu, Case Service tidak menulis langsung ke database Reporting Service.”

Catatan pendalaman. RabbitMQ merupakan pilihan desain, bukan syarat wajib semua microservices. HTTP langsung adalah alternatif dengan konsekuensi ketergantungan saat request berlangsung. Queue yang digunakan adalah kasus_status_updates. Pengiriman laporan awal memakai HTTP, bukan RabbitMQ.

8. Apa yang terjadi jika RabbitMQ atau salah satu service mati?

“Dampaknya bergantung pada komponen yang mati dan apakah event sudah diterima broker. Jika RabbitMQ mati saat publish, data kasus dapat tersimpan tetapi status laporan belum tersinkron. Jika Reporting Service mati, endpoint laporan tidak tersedia; event yang sudah tersimpan di broker dapat menunggu consumer kembali. Jika Case Service mati, login dan penanganan tidak tersedia, sedangkan pelaporan dapat tetap berjalan jika Reporting Service, gateway, dan MongoDB masih tersedia.”

Catatan pendalaman. Durable queue dan persistent message bukan jaminan bahwa event yang gagal dipublikasikan telah tersimpan. Kode saat ini mencatat kegagalan publish dan belum memiliki outbox serta publisher confirms. Startup Case Service juga menjalankan seed akun/master dan worker reminder, bukan startConsumer RabbitMQ; consumer status berada pada Reporting Service. Pernyataan dampak di atas merupakan penelusuran kode, bukan hasil uji kegagalan yang sudah dilakukan.

9. Apa kelemahan sistem Anda?

“Validasi aturan bisnis belum seluruhnya konsisten antara frontend dan backend. Pembatasan role dan urutan perubahan status juga perlu diperkuat. Sinkronisasi status masih berisiko tertinggal ketika pengiriman event gagal. Selain itu, gateway, broker, dan database masih menjadi titik ketergantungan bersama, file bukti memakai penyimpanan lokal, dan hasil evaluasi terbatas pada konfigurasi yang diuji.”

Catatan pendalaman. Pilih beberapa keterbatasan yang dapat dijelaskan beserta dampaknya. Contohnya, kronologi pendek dapat melewati validasi frontend melalui Postman; endpoint assessment/intervensi belum mengunci seluruh perubahan pada kasus arsip. Jangan menyatakan sistem sudah sepenuhnya aman atau tahan gangguan.

10. Jika penelitian dilanjutkan, apa yang pertama kali akan diperbaiki?

“Prioritas awal saya adalah memperkuat validasi dan kontrol akses pada backend, termasuk aturan transisi status dan penguncian kasus arsip, karena berkaitan dengan keutuhan data penanganan. Setelah itu saya memperkuat pengiriman event dengan outbox, konfirmasi broker, dan mekanisme retry yang terukur. Evaluasi lanjutan dilakukan dengan pengulangan serta konfigurasi resource yang lebih terkontrol.”

Catatan pendalaman. Ini adalah usulan pengembangan, bukan fitur yang sudah ada. Untuk penelitian performa, tetapkan skenario berdasarkan pertanyaan penelitian dan laporkan seluruh hasil; jangan memilih hanya pengujian yang membuat microservices terlihat unggul.

Menanggapi selisih performa yang kecil

“Selisih kecil tetap saya laporkan apa adanya. Saya memeriksa apakah hasilnya konsisten antarpercobaan dan apakah konfigurasi memberi ruang untuk menarik kesimpulan. Jika bukti belum cukup, saya membatasi kesimpulan pada hasil yang teramati dan mengusulkan pengujian lanjutan.”

Koreksi kesalahan hitung boleh dilakukan dengan menyimpan data mentah dan mencatat koreksinya. Mengubah angka untuk memperbesar selisih, menghapus hasil yang tidak disukai tanpa kriteria yang sah, atau menggambar grafik yang tidak sesuai data akan merusak validitas penelitian.

13 Pola menjawab saat sidang

Gunakan pola:

Keputusan → alasan → bukti implementasi/hasil → keterbatasan

Contoh:

“Saya menggunakan RabbitMQ agar Case Service tidak mengubah database Reporting Service secara langsung. Implementasinya menggunakan queue kasus_status_updates dengan publisher di Case Service dan consumer di Reporting Service. Konsekuensinya, perubahan status bersifat eventual consistency dan belum langsung terlihat pada waktu yang sama.”

14 Jawaban tambahan untuk pendalaman

Apa perbedaan service frontend dan microservice backend?

Service frontend adalah kumpulan fungsi JavaScript untuk memanggil API, misalnya laporanService.submitLaporan(). Microservice backend adalah aplikasi server tersendiri yang menjalankan logika bisnis dan mengakses database. Folder utils berisi fungsi bantuan seperti filter laporan dan notifikasi.

Apa fungsi token setelah login?

Token JWT menjadi bukti autentikasi untuk request berikutnya. Frontend atau Postman mengirimnya melalui Authorization: Bearer <token>. Backend memeriksa tanda tangan dan masa berlakunya, lalu menggunakan identitas serta role dari token. JWT ditandatangani, tetapi isi payload tidak otomatis dienkripsi. Masa berlaku default kode adalah 3600 detik dan dapat dikonfigurasi.

Apa perbedaan autentikasi dan otorisasi?

Autentikasi memeriksa siapa pengguna, sedangkan otorisasi menentukan tindakan yang diizinkan. Middleware auth memverifikasi JWT. Middleware role membatasi endpoint tertentu, misalnya pengelolaan pengguna untuk super_admin. Tidak semua route terproteksi sudah memiliki pembatasan role khusus.

Siapa yang boleh melakukan terminasi pada implementasi saat ini?

Pada alur tampilan, terminasi dilakukan melalui dashboard petugas. Di backend, route penanganan memakai middleware autentikasi, tetapi endpoint terminasi belum memiliki pembatasan role khusus. Karena itu jangan menyatakan hanya petugas yang dapat memanggilnya berdasarkan kontrol backend saat ini.

Apakah kasus yang selesai sudah sepenuhnya terkunci?

Terminasi mengubah status menjadi selesai dan arsip menjadi true. Penambahan log memeriksa arsip dan akan ditolak. Namun controller assessment dan intervensi belum memeriksa arsip secara menyeluruh, sehingga penguncian seluruh perubahan masih perlu diperkuat.

Bagaimana audit trail dan koreksi assessment ditangani?

Activity log menyimpan catatan, tanggal, nama petugas, dan timestamp. Ini belum merupakan audit trail lengkap untuk semua perubahan. Penyimpanan assessment saat ini dapat menimpa nilai sebelumnya tanpa riwayat versi; pengembangan berikutnya dapat menambahkan histori perubahan dan aturan koreksi.

Mengapa hasil Postman berbeda dari validasi formulir?

Postman langsung memanggil API tanpa menjalankan React. Validasi yang hanya berada di frontend dapat dilewati. Respons 422 untuk field wajib menunjukkan validasi model bekerja, tetapi belum membuktikan seluruh aturan bisnis sudah divalidasi di server.

Apa yang pertama diperbaiki jika penelitian dilanjutkan?

Prioritas teknis yang dapat diusulkan adalah memperkuat validasi backend dan akses berbasis role, kemudian memastikan event status tidak hilang dengan outbox atau mekanisme pemulihan yang teruji. Sesuaikan prioritas akhir dengan dampak masalah dan batas penelitian.

15 Lembar persiapan hasil pengujian

Lengkapi sebelum latihan sidang. Untuk hal yang tidak diukur atau tidak dilakukan, tulis secara jujur dan jelaskan dampaknya terhadap interpretasi hasil.

Judul dan rumusan masalah penelitian:

Metode penelitian dan alasan pemilihannya:

Endpoint dan payload pengujian:

Jumlah pengguna virtual serta ramp up:

Durasi atau loop dan jumlah pengulangan:

Spesifikasi perangkat serta lokasi load generator:

Batas resource per container dan total arsitektur:

Kondisi awal database dan prosedur reset setiap run:

Warm up dan urutan pelaksanaan pengujian:

Response time rata rata dan P95 beserta satuan:

Throughput dan error rate setiap skenario:

CPU dan memori jika diukur:

Kesimpulan utama beserta tabel atau grafik pendukung:

Batas generalisasi dan penelitian terdahulu pembanding:

Contoh pola jawaban hasil: Pada skenario [beban], arsitektur [nama] menghasilkan [metrik dan nilai], dibandingkan [nilai pembanding]. Temuan ini berlaku pada [konfigurasi pengujian]. Penyebab selisih [telah didukung pengukuran atau masih berupa dugaan].

16 Peta file untuk latihan demonstrasi

frontend/ircm-frontend/src/pages/LandingPage.jsx

Titik awal submit dan validasi frontend.

frontend/ircm-frontend/src/services/laporanService.js

Pemanggilan API laporan.

frontend/ircm-frontend/src/services/apiClient.js

Konfigurasi Axios dan header token.

nginx/nginx.conf

Routing menuju backend dan pemblokiran endpoint internal.

backend/reporting-service/app.js

Pendaftaran route dan koneksi report_db.

backend/reporting-service/src/routes/laporan.js

Route publik, Multer, dan route terproteksi.

backend/reporting-service/src/controllers/laporanController.js

Penyimpanan laporan dan cek status.

backend/reporting-service/src/models/Laporan.js

Skema, field wajib, kode laporan, dan status awal.

backend/case-service/src/controllers/kasusController.js

Registrasi sampai terminasi.

backend/case-service/src/queue/publisher.js

Pengiriman event status kasus.

backend/reporting-service/src/queue/consumer.js

Penerimaan event dan pembaruan status laporan.

backend/case-service/src/controllers/authController.js

Login dan pembuatan JWT.

docker-compose.yml dan docker-compose.pengujian.yml

Service, database, port, dan konfigurasi eksperimen.